A Mixture-of-Experts Decode Layer at 2.25× SGLang on an H200: An Experience Report from the Open Kernel Challenge

AUTHOR NAME
github.com/yelgabo/okc

November 5, 2026

Abstract

The Muse Spark Open Kernel Challenge scored entrants on one GPU's share of a Mixture-of-Experts transformer decoder layer, timed on an NVIDIA H200 against SGLang's production kernels across twelve shapes. Over nine days (27 October to 4 November 2026) one person ran a team of LLM coding agents on this problem. Our final file averages 2.250× SGLang over nine judged runs. Its best run, 2.280× with a 95% interval of [2.245, 2.314], was our board row and ranked 153rd. The mean equals, within the judge's interval, the 2.249× that the best public layer file measured on our account. We forked that file on 1 November. Our own safety audit of the fork switched off three of its features whose correctness we could not prove. Most of our later gain came from proving those features correct and switching them back on: programmatic dependent launch across kernel boundaries, and per-expert completion counters in place of a grid barrier in the routed-expert kernel. We describe the layer, the timing rules, what we changed and what we inherited, the variants that lost, an organizer disqualification of a routing optimization we had read as allowed, and the process failures of an agent team working against a rationed judge. Every number has a source in a public repository.

1 Introduction

GPU kernel competitions now attract agent-driven entries. The Open Kernel Challenge [1] posed five problems drawn from one model, Muse Spark Open. One of them, the whole decoder layer for batched decode and speculative verification, decided each team's overall rank.

This report covers one entry. The work ran from 27 October to the close of kernel submissions at midnight Pacific on 4 November 2026. One person steered. LLM coding agents wrote nearly all of the code, ran the experiments and kept the records; most were Claude Code sessions, with some Codex sessions early on. The repository holds 2,646 commits, a ledger of every judged run and a list of about 445 ruled-out variants with their measurements [3].

The result is modest. The board row placed 153rd, about 1% below the 100th-ranked entry, and the final file's mean speed matches the public file it descends from. We still think the record is worth publishing, for three reasons:

- It measures where time goes in a well-optimized MoE decode layer at small batch, and which plausible optimizations lose there.
- It gives a concrete case of a synchronization lesson. A grid-wide barrier in the routed-expert kernel could be replaced by per-expert completion counters, once someone proved the replacement safe and stopped treating a disabled feature as settled.

- It records how an agent team failed: stale clocks, shared checkouts, a measurement flag that changed the thing measured, and a routing optimization that passed every numerical check and was then disqualified.

2 The benchmark

2.1 What one step computes

The problem is everything one GPU does for one layer under 8-way tensor parallelism and 8-way expert parallelism, with all-reduces treated as the identity [2]. With x the attention input ($T \times 8192$, BF16) and s the FP32 residual stream:

$$
\begin{aligned}
q, k, v, g &= \mathrm{split}(xW_{qkvg}^{\top});\ \text{QK-norm, RoPE, write fp8}(k), \text{fp8}(v) \\
a &= \big(\mathrm{rmsnorm}(\mathrm{attn}(q)) \odot \sigma(g)\big) W_o^{\top} \\
s &\leftarrow \alpha_1 s + \beta_1\, \mathrm{rmsnorm}(a), \qquad h = \mathrm{rmsnorm}(s) \odot (1 + w) \\
E &= \mathrm{top8}\big(\sigma(0.177\, hW_r^{\top}) + b\big) \\
y &= \textstyle\sum_{e \in E \cap \mathrm{local}} \mathrm{expert}_e\big(\mathrm{rmsnorm}(hW_{pool}^{\top}) \odot (1 + w)\big) \\
f &= yW_{unpool}^{\top}, \qquad s \leftarrow \alpha_2 s + \beta_2\, \mathrm{rmsnorm}(f \odot (1 + w))
\end{aligned}
$$

The kernel returns $\mathrm{rmsnorm}(s) \odot (1 + w)$, the new s and the chosen experts. This GPU holds 16 query heads and 2 KV heads of dimension 64, so W_{qkvg} is 2304×8192 and W_o is 8192×1024. The router scores 256 experts in FP32; this GPU owns 32 of them. A pooler projects to width 4096 before the experts and an unpooler projects back. Each expert is a SwiGLU block at width 4096 whose weights are NVFP4: E2M1 values, one FP8 scale per 16 values and one FP32 scale per expert. Activations stay BF16 wherever SGLang keeps them. The KV cache is FP8 E4M3, and queries may be rounded to E4M3 as FlashAttention-3 does.

2.2 Shapes, timing and score

There are six loads: decode at batch 1, 16 and 32, and speculative verification of an 8-token draft at batch 1, 16 and 32. The token counts are $T = 1, 16, 32$ and $T = 8, 128, 256$. Each load times one sliding-window layer (window 2,048, weight 46) and one full-attention layer (weight 16), both at a context of about 80,000 tokens. A load's time is the weighted sum of its two shapes. The score is the geometric mean over loads of SGLang's time over ours, so each load counts equally. The board shows a time as well: SGLang's fixed per-step time of 22,620.3 µs divided by the speedup.

The judge captures the call as a CUDA graph and replays it. Every replay starts with a cold L2, and our replays alternate with SGLang's on the same fresh inputs. A shape's speedup is the median of paired ratios, and a run's score carries a 95% interval of at least ±1.5%. The baseline is SGLang at commit `a55ab7d5`: cuBLAS for the dense projections, FlashAttention-3 for attention, and Marlin W4A16 GEMMs for the experts. Before timing, a run must pass 13 correctness tests, some at contexts up to 2^{20} tokens, and every timed call is checked against an FP32 reference.

The rules matter as much as the math. Weights may be repacked losslessly but not stored at a lower precision. Every key in the window or context must be attended to. Nothing derived from the inputs may be cached. Routing may differ from the reference only on near-ties, within stated tolerances. Each problem allowed a daily number of public runs and of private "check" runs, and identical source submitted twice returns its earlier result.

Table 1: The twelve timed shapes on the final file's best judged run (`908cc07e43364008`). Times in µs. The floor is the judge's speed-of-light bound; "of floor" is our time over it.

load	layer	SGLang	ours	speedup	floor	of floor
decode 1	sliding	170.77	65.84	2.594	40.02	1.65
decode 1	full	180.74	71.66	2.522	44.18	1.62
decode 16	sliding	268.51	127.15	2.112	76.72	1.66
decode 16	full	439.92	202.29	2.175	137.46	1.47
decode 32	sliding	261.92	141.18	1.855	84.51	1.67
decode 32	full	621.41	292.72	2.123	208.10	1.41
verify 1	sliding	260.02	99.91	2.603	62.29	1.60
verify 1	full	281.15	113.22	2.483	66.44	1.70
verify 16	sliding	507.90	233.53	2.175	126.61	1.84
verify 16	full	672.82	311.93	2.157	188.68	1.65
verify 32	sliding	701.26	298.79	2.347	143.81	2.08
verify 32	full	1071.82	466.19	2.299	267.18	1.74

Table 1 lists the twelve shapes with SGLang's time, our final file's time on its best run, and the speed-of-light floor. The floor is the time to move the data at 4.8 TB/s or to do the math at 989 TFLOP/s, whichever is longer.

2.3 Where the baseline spends its time

Profiles of the SGLang baseline show three regimes. At decode 1 on a sliding layer (170.0 µs), no single kernel dominates. The Marlin expert GEMMs take 40.8 µs (23%) and a split-K cuBLAS projection 31.4 µs (18%); the rest is a long chain of small launches. At decode 32 on a full layer (610.1 µs), FlashAttention-3 takes 68% reading the 80K-token FP8 cache for 32 requests. At verify 32 on a sliding layer (676.7 µs), Marlin takes 73%. A competitive entry needs all three: a short launch chain at $T = 1$, an attention kernel that streams the cache at close to full bandwidth, and an expert kernel that reads each active expert's NVFP4 weights once.

3 How the work was organized

3.1 People, agents and machines

One person (the author) set goals, approved public submissions and decided rule questions. A lead agent session steered and kept the ledger. Worker sessions each owned one "lane" named by its job (attention, dense projections, MoE, programmatic dependent launch, layer fusions, box efficiency) and worked in its own git worktree. For the last afternoon we also ran EAT [6], an open-source supervisor that drives a team of Claude Code agents against a scripted evaluation, with a $400 budget. It spent $131.94 and produced the final file's last change.

Official numbers came only from the challenge's judge: `okc check` for private runs and `okc submit` for public ones. The quota started at 10 public runs and 20 checks per problem per day and later rose to 20 and 40. On the last day each job waited 15 to 20 minutes in the queue. From 3 November we also rented an H200 SXM on RunPod at $5.29 per hour for paired A/B timing against our own public file. The rented box ran short loads 9–18% faster than the judge, so we used it only to rank two files against each other, never to predict a judge number.

3.2 Where the code came from

Our own line of files, built from kernels we first wrote for the three standalone component problems, scored 1.168× on its first judged check on 28 October, 2.014× on 30 October and 2.043× (rank 373) on 1 November. On 2 November the tenth-ranked entry stood at 2.422×.

On 1 November the author approved forking the best public layer file, published by the team keell0renz [4]. That file built on alygg77's entry, which in turn built on vjeux's [5]; it also carried code credited to chenwainuo, Matsvei and zhebrak. The challenge terms license every submission under Apache-2.0, and our final file keeps every attribution notice. The unmodified donor measured 2.249× [2.215, 2.283] on our account (job `3a5f879df0d44825`). We never submitted it.

Before using the fork we audited its synchronization. Where we could not prove an ordering or progress guarantee, we switched the feature off. The audit turned off layer-wide programmatic dependent launch, the expert kernel's dependent launch, and the per-expert completion counters that let the expert kernel skip a grid barrier. It also added a BF16 rounding of the attention output and a fallback for empty routing. That first fork scored 2.126× (rank 303). The ledger does not price the audit's changes one by one.

3.3 Rules the team adopted

Every experiment registered its hypothesis, control, affected shapes and acceptance threshold before dispatch. Every loss went into the ruled-out list with its numbers, and reopening an entry required new evidence. A box A/B needed at least four paired rounds with the judge's launch configuration before a change counted as a win or a null. Public runs needed the author's go-ahead. Section 7 gives the failure behind each rule.

4 What we inherited and what we changed

The final file is `eat_b4.py`, one 21,117-line Python file that compiles its CUDA sources at preparation time. Gains marked "box" are paired medians on the rented H200. They are not additive, and most are smaller than the judge's run-to-run spread, a standard deviation of 0.78% over the final file's nine runs.

4.1 Inherited structure

Most of the final file's design came with the fork. Below 128 tokens, the dense projections run as GEMV kernels ("sk2") that stream weights through a TMA and mbarrier ring, with a lossless 12-bit encoding of the BF16 weights and exact escape correction. These kernels already issued their own dependent-launch triggers. Attention is one fused launch that does QK-norm, RoPE, the FP8 cache write, split-KV attention, the merge and the gated RMSNorm. The splits merge through `red.global.add` into a fixed frame, so no split waits for another's running maximum. The routed experts run as one cooperative launch per step. Two TMA producer warps feed four WGMMA warpgroups, NVFP4 weights decode to BF16 in registers, and a cost table assigns work stream-K. Block scales are stored losslessly in 6 bits. Preparation times each backend in graph replay and dispatches per token count. The per-expert completion counters (below) were in the donor file too.

4.2 Programmatic dependent launch, proved edge by edge

Programmatic dependent launch (PDL) lets a kernel start while its predecessor is still running. The consumer executes `griddepcontrol.wait` before it reads the producer's output, and the pro-

ducer signals early with `griddepcontrol.launch_dependents`. Work that does not depend on the producer, such as loading weights, can run before the wait.

After the audit switched PDL off, we brought it back one edge at a time. Each edge needed a written argument about the producer's publication, the consumer's wait and, for cooperative grids, residency and progress. A 2,000-replay drift guard runs in preparation and can turn an edge off. The enable set itself is fixed at build time, because an earlier gate that chose edges by timing during preparation gave four different enable sets across 35 logs of the same bytes. Two edges around attention produced a 6×10^{-3} output drift at $q_{\text{len}} = 8$ and stayed off at $T = 8$ until a later file re-enabled all six edges behind the guard.

PDL was the largest single gain after the fork. The public score went from 10,639.4 to 10,419.9 µs with PDL at $T = 1, 16, 32$, and to 10,374.6 µs with most edges on at $T = 8$. In the final file's preparation log, PDL saves a median 6.51, 7.82, 7.77 and 6.80 µs at $T = 1, 8, 16, 32$. The last edge, the expert kernel as a dependent of the routing kernel, gained less than the others. The driver does not overlap a cooperative launch, so that edge only shortens graph launch latency, from 0.76–0.96 to 0.38–0.50 µs.

Our first PDL attempt, on 28 October in our own line, triggered every kernel at entry and gained nothing measurable at decode 1 (113.1 to 115.4 µs).

4.3 Per-expert completion counters instead of a grid barrier

In the version we ran from 1 to 4 November, the routed-expert kernel ended with a grid-wide barrier between the expert GEMMs and the final pass, which weights and sums each token's expert outputs. The final pass for a token needs only that token's experts to be finished. The donor's alternative does exactly that. Writers release a per-expert completion counter after each unit's epilogue (`red.release.gpu`), and the final pass acquire-polls (`ld.acquire.gpu`) only the counters of its own token's experts. Our audit had forced this path off, because the poll gave up after four million iterations and then read unreduced data.

On 4 November a top-ten team's write-up, relayed by the author, described removing this barrier. Two two-line patches followed. The first re-enabled the counters on the 17–32-token build and removed the poll's cap. The second extended them to the build for larger T. A ruled-out entry written at 13:19 that day called that build's barrier necessary "by construction", because its finishing CTAs supposedly never released the counters. A re-read of the code reversed it at 13:43. Removing the cap trades a silent wrong answer for a possible hang. The co-residency of a cooperative launch rules out a hang only if every counter target is reachable, and the ledger records that as a remaining risk. No hang appeared in any judged or box run.

The box measured decode 32 at $-1.75/-1.67$ µs (sliding/full) for the first patch and verify 32 at $-1.59/-1.40$ µs for the second, with numerics bitwise unchanged. On the judge, the first patch's check scored 10,046.1 and then 10,070.4 µs against 10,109.9 µs for its parent, and the decode 32 paired ratios rose from 1.801 to 1.846 (sliding) and 2.063 to 2.194 (full). With the second patch added, two checks drew 10,111.6 and 10,101.5 µs. At this noise level the judge cannot separate the second patch from zero.

The counters had stayed off for three days. One reason is a final-day rule that ranked work by score gain per microsecond on the short loads to ration judge checks. The barrier's cost sits in the $T \geq 16$ cells, which that rule pushed down the list.

4.4 Smaller changes of ours

- At $T = 8$, 128 and 256, the expert kernel's finishing CTA keeps its own share of the sum in registers instead of writing and re-reading it. Box: verify 32 sliding -2.75 µs, faster in 54 of 54 pairs.
- A publication-ordering fix in the expert kernel's hand-off between phases. It was neutral on the score.
- The unpooler runs as a PDL dependent of the expert kernel at $T = 8$, 16 and 32 (box: -0.97 to -2.06 µs), and its post-wait activation loads go through L1 (box: -0.67 µs at decode 1).
- The $T = 1$ QKV projection moved to the narrow GEMV build already used for $T = 2$ to 8 (box: $-1.32/-0.37$ µs).
- At $T = 128$ an in-house GEMV replaced cuBLASLt for QKV, with attention as its PDL dependent. Evidence is mixed. The final file's preparation log measures $-1.31/-1.86$ µs, while a box A/B measured $+0.72/+1.33$ µs on the sliding cells.
- Requesting 81,920 B of shared memory puts two decode 16/32 sliding attention CTAs on each SM (box: -1.19 µs at decode 32 sliding). Requesting 118,784 B for the $T = 8$ finish kernel spreads it to one CTA per SM (box: $-0.41/-0.32$ µs).
- EAT's change, the last one, computes the first residual's per-row sum of squares across a thread-block cluster. CTAs write into a peer's shared memory with `st.async` and count arriving bytes on an mbarrier, and the kernel fires its PDL trigger at entry (box: -0.3 to -1.0 µs at decode 1, verify 1 and decode 16).

One constraint shaped the last day. Submissions were capped at 1,000,000 bytes, and one stacked file came to 1,057,604. We stripped its comments, keeping legal notices and runtime anchors, and checked that the program's syntax tree was unchanged.

4.5 Where the time still goes

Table 1 shows the final file at 1.41–2.08 times the floor. By share of profiled GPU time, the routed-expert kernel is the largest kernel in every cell except three. Attention leads at decode 16 and 32 on full layers and is about even with the experts at verify 32 full (179.6 against 175.4 µs). At decode 1 sliding the expert kernel takes 32% of profiled time (22.5 µs), and the three projection GEMVs take 32.5 µs together. Our weakest cell is decode 32 sliding at 1.855×, where the expert kernel takes half of profiled time (73.8 µs). On the verify 16 and 32 sliding cells, cuBLASLt projections still take 55 to 87 µs per cell.

5 Results

Figure 1 shows every judged layer result that set a new best, plus the two disqualified runs. Table 2 in the appendix lists them with job ids. The score rose in two phases. Our own line went from 1.168× to 2.043× in five days, mostly by porting our standalone kernels into the layer: the fused W4A16 expert kernel, the one-launch attention and the GEMV projections. The fork started at 2.126×, and three days of work on it took the best run to 2.280× and the mean to 2.250×.

The final file ran nine times on the judge, three public runs and six private checks. They span 9,922.9 to 10,160.8 µs, with a mean of 10,054.0 µs (2.250×) and a standard deviation of 0.78%. The best of them, run `908cc07e43364008`, is the board row: 2.2796× with a 95% interval of [2.2454,

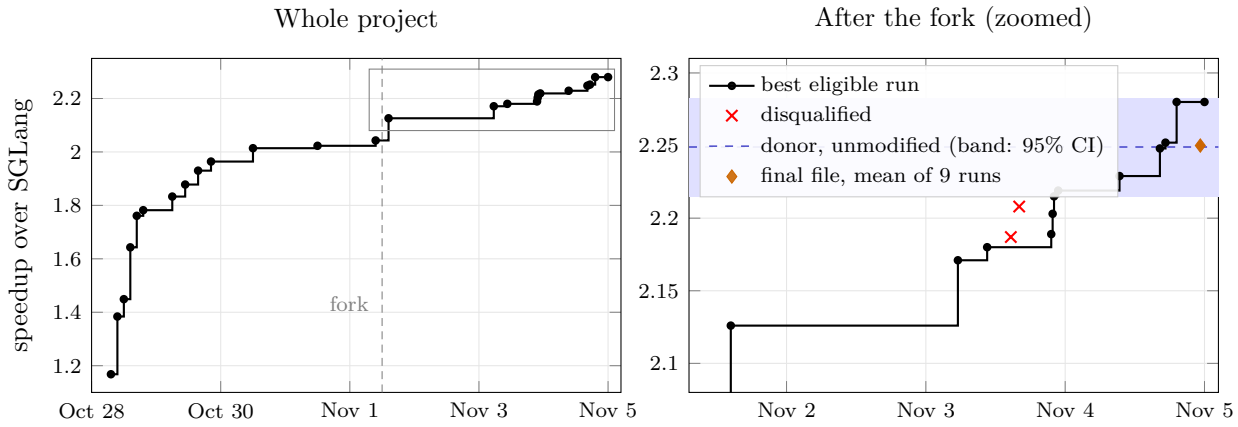


Figure 1: Best eligible decode-layer result over time, official judge only. Left: the whole project; before the fork the points are our own line. Right: the boxed region enlarged. The unmodified donor file was measured once on our account (2.249×, shaded 95% interval); the final file's mean over nine runs falls inside that interval. Rows on 28–31 October have dates but no recorded times and are spread within their day.

2.3139]. The judge returns a cached result for identical bytes, so each repeat differed only in a nonce. The best run is 1.3% faster than the mean. That selection effect is part of the board position, and entries are better compared by their means.

A later EAT file with one more attention build flag had a slightly lower eight-run mean (10,029.6 µs), but none of its runs beat the row. At 22:36 Pacific on 4 November the row ranked 153rd of the layer board, which had 1,222 entries on 2 November. At the close the team stood 158th on the overall board. At 18:28 Pacific the 100th-ranked entry was at 2.3028×.

Per load (Table 1), the best run's speedups are 2.574 (decode 1), 2.134 (decode 16), 1.967 (decode 32), 2.569 (verify 1), 2.169 (verify 16) and 2.330 (verify 32). Their geometric mean is 2.2796, the reported score.

The standalone boards, ranked separately, ended at 2.3893× for the routed-expert kernel (rank 258), 2.6043× for attention and 1.5321× for the dense projections.

6 What did not work

These are the most instructive entries in the ruled-out list, with the numbers that closed them.

L2 prefetch of later weights during attention. Prefetching the 34 MB of o, router and unpooler weights beside attention cut the o GEMV from 7.3 to 5.9 µs, but the whole step got 3.5–4.1 µs slower at decode 1 in the paired comparison. The prefetch traffic cost more than the hits saved. Several other teams reported the same in public.

Fusing across stage boundaries. One cooperative launch for the o projection, the attention boundary and the pooler/router saved 0.5–1.3 µs inside the process and nothing on any short load. Fusing the experts into the unpooler made decode 1 worse (80.7 against 76.8 µs); it replaced two launch boundaries with five dependent global round trips. A cooperative o plus boundary kernel was 1.16 µs slower in 9 of 9 rounds, from which we estimate a grid barrier at about 1 µs.

Using idle SMs. The QKV GEMV launches 96 CTAs on 132 SMs. A stream-K version with 128 one-wave CTAs at $T = 8$–32 lost 2.9 to 4.0 µs to the join alone, and the variant with no reduction

lost too. Idle SMs were not what limited this kernel.

More concurrency in the expert kernel. Two expert CTAs per SM lost 7.9/8.5 µs at decode 1, through a doubled per-CTA prologue, grid barrier and final work. A four-slot weight ring lost 2.0/2.2 µs through slower hand-off and a wider finish. A second N-tile, meant to avoid re-streaming weights at decode 16, lost about 10 µs, the third loss on that idea. In a math-only measurement at $T = 8$ and 32, the expert kernel's unit loop sat at an issue-throughput ceiling of about 0.5 warp-instructions per cycle per SM sub-partition, the same at four and six consumer warps. If that holds in the full kernel, the next gain there needs fewer instructions per weight word.

Synthetic routing. We first chose expert-kernel variants on synthetic routing, where a 1–3 µs margin on two seeds decided a 30 µs question. With routing skewed the way the judge's is, verify 16 dropped 25–30 µs.

Porting layer wins back to the standalone problems. Projection and attention choices tuned inside the layer gave nothing on the standalone boards. Only the expert-kernel port helped there, by about 0.7% on the box. We did not find out why.

6.1 The routing-slack disqualification

The rules let routing differ from the reference on near-ties. Each chosen expert has to score within 0.05 of the reference's eighth best, and clear winners must be kept [2]. On this GPU each distinct local expert costs the fused kernel about 6.3 µs of weight reads. So at $T = 1$ we swapped near-tied local experts for near-tied experts on other GPUs, which this GPU then never reads. Decode 1 got about 5 and 3 µs faster on sliding and full layers. The file passed every correctness test. The author approved it under the written tolerances, and it went public at 10,343.1 µs, rank 355.

On 3 November at 17:45 Pacific the organizers disqualified both public runs of the technique, and we withdrew a third. Their reason was that the change "swaps near-tied experts for ones on other GPUs so their weights are never read, which changes the model's output" and so "goes against the spirit of the challenge". The row fell back to 10,374.6 µs (2.180×, rank 364). Every file built on the slack file had to be rebuilt from its slack-free parent and re-qualified. A review confirmed that the routing itself was restored, beyond checking that the slack code was gone. Our promotion audit had also counted a disqualified source as clean, because it read the numerical pass flags and not the separate disqualification fields. A tolerance written to absorb a reference's rounding is not permission to choose outputs for speed, and passing the checks is not the same as being eligible.

7 Process lessons

An agent team fails in ways a single engineer usually does not. Each failure below cost real time and ended in a written rule.

Estimated clocks. On the last day the lead agent stamped ledger times from its own estimates and ran 3.5 hours ahead of the real clock. Separately, a plan written that morning said 11.5 hours remained when about 13.5 did. Rule: every ledger or schedule time comes from `date` in the same command.

Shared checkouts. Another session's merge reset the working tree and wiped staged edits, which we recovered with `git fsck --dangling`. A union merge of a CUDA source doubled a pipeline stage-advance line, and that line was pushed. Rules: edit, stage, commit and push in one call; resolve source conflicts by hand; give every branch its own worktree.

A flag that changed the experiment. A box script's `--skip-pdl8-gate` flag also turned off the expert kernel's PDL edge. With the flag, one change read faster in 7 of 7 pairs; without it, it read null over 3 pairs. We built the next file without that change. Four pairs with the edge on later showed it 0.48/0.52 µs faster. Rule: measure with the configuration the judge uses, in at least four pairs.

Rationing that narrowed the search. The rule that ranked work by gain per microsecond on the cheapest loads was meant to ration judge checks. It also narrowed what the agents looked at, and the completion-counter fix sat in the excluded cells. Rule: rank engineering work by its effect on the whole score, and treat a feature compiled in but switched off as a lead to re-check.

Context limits. Lane sessions reached 250,000–500,000 tokens per round and became slower, costlier and less reliable. Rule: each lane writes a checkpoint file after every step and restarts fresh after about 60 tool calls.

An async judge. With 15–20 minute queues, a session that waited on a job did nothing else. Rule: submit with `--no-wait`, record the job id in the ledger and wait in the background.

8 Discussion

Our numbers support a plain reading. The fork's inherited design carried the score. Our work after the fork took the file from 2.126× to a mean of 2.250×, and against the unmodified donor measured on our account that gain is zero within the judge's interval. What the work bought is the same speed with written arguments for ordering and progress on every dependent-launch edge and on the completion counters, which the donor shipped without. We did not show that the donor was wrong anywhere. Whether those arguments are worth the time is a fair question for anyone running a short competition.

Two observations may help others working at small batch on this GPU. With PDL, a kernel boundary inside a CUDA graph cost us less than the synchronization that the cross-stage fusions we tried put in its place. Fusion inside a stage, such as the one-launch attention kernel, still paid. Second, a feature that someone switched off for safety deserves a re-check against the code once the pressure is gone. Ours held the largest single fix of the last day.

For agent-driven kernel work, the largest gains came from reading: a public file, a top team's write-up, a disabled code path. Local tuning gave the smallest. Measurement discipline mattered most where it was weakest. A box script measured a different configuration than the judge, a quota rule hid the best remaining fix, and a numerical tolerance was read as permission.

Use of AI tools

LLM coding agents wrote most of the code and kept the experiment records: Anthropic's Claude models through Claude Code, OpenAI Codex sessions, and the EAT supervisor running Claude Code agents. A Claude agent drafted this report from the repository's ledgers, and a second agent checked its claims against the sources. The author chose the work, approved every public submission, and is responsible for the content.

Table 2: Decode-layer results that set a new best, plus the disqualified runs. Times are the board's: SGLang's fixed 22,620.3 µs over the speedup. Times marked ∼ are derived that way because the ledger records only the speedup. "check" is a private run; DQ means disqualified. Sources: `docs/runs-decode-layer.md` and `docs/steering/2026-11-02-final-day-directive.md`.

date (PT)	job	change	µs	speedup	rank
Oct 28	`c61c26ce`	MoE port only (check)	∼19,366.7	1.168	
Oct 28	`6c9eb0ed`	+ W4A16 expert GEMMs (check)	∼16,344.1	1.384	
Oct 28	`b216cf4a`	cluster residual kernel (check)	∼15,611.0	1.449	
Oct 28	`f23f1c81`	fused MoE, GEMV projections	∼13,767.7	1.643	
Oct 28	`cc7a07e2`	one-launch attention	∼12,845.1	1.761	
Oct 28	`a41ace0b`	MoE ordering repair	∼12,693.8	1.782	
Oct 29	`344ae513`	judge-like MoE routing	∼12,340.6	1.833	
Oct 29	`5b95c24e`	attention trimming	∼12,044.9	1.878	
Oct 29	`0fae6468`	MoE schedule mode 28	∼11,720.4	1.930	
Oct 29	`9b921feb`	routing front end	∼11,517.5	1.964	
Oct 30	`3fbb3ad0`	clean release, mode 47	∼11,231.5	2.014	
Oct 31	`8f2780d7`	attention port	∼11,182.3	2.023	
Nov 1	`cb71acec`	residual boundary	∼11,069.7	2.043	373
Nov 1	`607eb52b`	audited fork of public file	10,639.4	2.126	303
Nov 3 05:25	`fefd7f7b`	PDL at $T = 1, 16, 32$	10,419.9	2.171	369
Nov 3 10:40	`f931abf5`	PDL at $T = 8$	10,374.6	2.180	364
Nov 3 14:40	`9d80ae24`	routing slack (DQ)	10,343.1	2.187	355
Nov 3 16:07	`86ed5dd3`	routing slack repeat (DQ)	10,243.9	2.208	
Nov 3 21:32	`117605d0`	clean rebuild	10,333.4	2.189	367
Nov 3 21:39	`a7741ba2`	expert-kernel micro changes	10,267.5	2.203	353
Nov 3 22:09	`1798d468`	$T = 16$ residual on a per-row kernel	10,211.3	2.215	322
Nov 3 22:43	`7d03f88d`	follow-up to the previous row	10,194.1	2.219	311
Nov 4 09:25	`caed0290`	finisher in registers	10,146.4	2.229	308
Nov 4 16:20	`40ac6a24`	expert kernel as PDL dependent	10,061.9	2.248	
Nov 4 17:16	`5dbfa4b0`	completion counters, both builds	10,044.3	2.252	246
Nov 4 19:07	`908cc07e`	EAT cluster residual (final)	9,922.9	2.280	153

A Judged layer runs

References

[1] Muse Spark Open Kernel Challenge. `https://openkernelchallenge.dev`, rules at `https://openkernelchallenge.dev/rules`, 2026.

[2] Muse Spark Open Kernel Challenge. Problem statement for `muse_decode_layer`, `TASK.md` written by `okc init` 0.7.2, 2026.

[3] This work's repository, with run ledgers, profiles and the ruled-out list. `https://github.com/yelgabo/okc`.

[4] keell0renz. Muse open kernel challenge entry and `LESSONS.md`. `https://github.com/keell0renz/muse-open-kernel-challenge`, 2026.

[5] vjeux. OKC 2026 submissions. `https://github.com/vjeux/okc-2026`, 2026.

[6] caoshiyi. EAT. `https://github.com/caoshiyi/EAT`, 2026.